Ball-by-Ball Win Probability for Chasing Teams in Men's T20 Cricket World Cup

A Dissertation for

Course code and Course Title: CSD-651 Research Project on Data Science in Academic or Research Institutes or Industry

Credits: 16

Submitted in partial fulfilment of Masters Degree

MSc. in Data Science

by

PAVAN PURSHOTTAM SANNAIK

Seat No.: 24P0630010
ABC ID: 993249231473
PRN: 202100534

Under the Supervision of

SHRI JARRET STEVAN ANTHONY FERNANDES

Goa Business School
Computer Science and Technology

GOA UNIVERSITY

APRIL 2026

Examined by: Seal of the School

DECLARATION BY STUDENT

I hereby declare that the data presented in this Dissertation report entitled, "Ball-by-Ball Win Probability for Chasing Teams in Men's T20 Cricket World Cup" is based on the results of investigations carried out by me in the Discipline of Computer Science and Technology at Goa Business School, Goa University under the Supervision of Shri Jarret Stevan Anthony Fernandes and the same has not been submitted elsewhere for the award of a degree or diploma by me. Further, I understand that Goa University or its authorities will not be responsible for the correctness of observations / experimental or other findings given in the dissertation.

I hereby authorize the University authorities to upload this dissertation on the dissertation repository or anywhere else as the UGC regulations demand and make it available to any one as needed.

Mr. Pavan Purshottam Sannaik

Seat no.: 24P0630010

Date:

Place: Goa University

COMPLETION CERTIFICATE

This is to certify that the dissertation report "Ball-by-Ball Win Probability for Chasing Teams in Men's T20 Cricket World Cup" is a bonafide work carried out by Mr Pavan Purshottam Sannaik under my supervision in partial fulfilment of the requirements for the award of the degree of Masters of Science in Data Science in the Discipline Computer Science and Technology at Goa Business School, Goa University.

Shri Jarret Stevan Anthony Fernandes
(Assistant Professor)

Date:

Signature of Dean of the School/HoD of Dept

School/Dept Stamp

Date:

Place: Goa University

CONTENTS

CONTENTS (Continued)

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Shri Jarret Stevan Anthony Fernandes, for his invaluable guidance, continuous encouragement, and technical insights throughout the duration of this research. His mentorship played an important role in navigating the complexities of cricket analytics and deep learning structures.

I am also thankful to the faculty members of Goa Business School for providing a good environment for academic research. Finally, I thank my family and friends for their unwavering support and patience during the completion of this dissertation.

TABLES

FIGURES

ABBREVIATIONS USED

Entity	**Abbreviation**
Comma-Separated Values	CSV
Current Run Rate	CRR
Deep Learning	DL
Duckworth-Lewis-Stern	DLS
Exploratory Data Analysis	EDA
Gated Recurrent Unit	GRU
Indian Premier League	IPL
Long Short-Term Memory	LSTM
Machine Learning	ML
Most Valuable Player	MVP
Multi-Layer Perceptron	MLP
One Day International	ODI
Ordinary Least Squares	OLS
Radial Basis Function	RBF
Receiver Operating Characteristic Area Under the Curve	ROC AUC
Recurrent Neural Network	RNN
Required Run Rate	RRR
Support Vector Regression	SVR
Twenty20	T20
Yet Another Markup Language	YAML

ABSTRACT

In the intense world of T20 cricket, predicting the outcome of a match as it unfolds is a complex but an extremely valuable task. This dissertation aims to create a reproducible pipeline to calculate ball by ball win probabilities for chasing teams in the Men's T20 World Cup matches by implementing various models on the dataset. By using data from the Cricsheet website starting from 2007 to 2025, this study addresses significant data engineering issues such as keeping accurate track of deliveries bowled, creating unique Match ID to handle large volume of tournament data without errors and using various key features of the dataset to compute win probability for the chasing team per ball bowled. This is then applied to the matches to get the better understanding of the matches being played.

While the research examines a range of predictive methods, it focuses mainly on statistical models, machine learning and deep learning architectures. The findings show that the recurrent networks such as Gated Recurrent Units and Long Short Term Memory provide far better accuracy and performance by learning how previous match events influenced future deliveries.This method enables us to understand the game better than the ordinary linear approaches. The project ends with a flexible framework that should be used to include new matches of the 2026 Men's T20 World Cup as good a tool in real world sports analytics and team decision support.

Keywords: T20 World Cup, Win Probability, Ball-by-ball, Deep Learning, LSTM, GRU, Sports Analytics.

1. INTRODUCTION

1.1 BACKGROUND

Over the past twenty years, cricket has radically changed, becoming not only a sport dominated by tradition and intuition of the players and coaches, but also one of the most data-driven professions in the world of sport. The Twenty20 (T20) format lies at the heart of this change. Being marked by quick changes in momentum, T20 cricket requires players and support staff to decide in split seconds. In contrast to the traditional Test or One Day International systems, where the game is played over days or even hours, a single delivery in a T20 game can be a game changer as it is either a boundary or a wicket or even a non-legal delivery that can make one side a winner or the other one a loser.

The incorporation of analytics in live broadcasting of sports has transformed the way people watch and plan cricket. Fans, coaches and commentators can not base their judgment only on subjective judgments; they desire real-time, data-driven information that quantifies and measures the game flow. Win probability models have grown to be part of this new viewing experience and it gives a continuous, objective indicator of which team is on the winning side. But the T20 format is notoriously hard to model because it has a compressed timeframe and traditional statistical anchors tend to miss sudden violent changes of momentum.

Additionally, the nature of a T20 match is also different in the second innings. A pursuing team is one which has the pressure of scoreboard on it at all times. The chase, unlike the first innings, is a resource management exercise, the goal being a closed-ended maximization of the number of runs. The batting team must also continually juggle the growing required run rate with the number of wickets left and deliveries available. This deterministic goal provides a special mathematical setup in which each individual ball either raises or lowers the mathematical probability of winning explicitly.

The motivation behind this research is my personal interest in the statistics of the cricket game and the desire to work in the sphere of sports analytics on a professional level.

The idea behind the project is that it is a strictly statistical project to investigate the match conditions using the available info. The Men's T20 world cup is the culmination of this format, providing a proper standardized and highly competitive setting. The world cup, in contrast to other franchise leagues, offers a steady platform to study elite performance in extreme conditions, where there is a greater quality of competition and player talent.

Although the format has gained popularity across the globe and commentators and sports teams are eager to generate real-time insights, it is still a technical challenge to develop accurate, ball-by-ball win probability signals, even though they are in high demand. This calls on the need to chnage the traditional methods with the more complex, data-oriented predictive models.

1.2 STATEMENT OF THE PROBLEM

The main technical goal of this study will be to generate calibrated ball-by-ball probabilistic forecasts to the chasing team. Conventional techniques such as the Duckworth-Lewis-Stern (DLS) technique or simple Current Run Rate (CRR) predictions are not always adequate in ball-by-ball analysis since they do not consider the fluid mechanics of a T20 run pursuit.

One of the most important issues in this area is that there is hardly any good, cleaned data that properly takes into consideration the special rules of sport. The information found in cricsheet website [1], needs to be transformed into an appropriate form where new columns will need to be created manually based on the already existing data i.e.: remaining balls, current run rate of batting team , required run rate for chasing team, cumulative runs scored per ball, cumulative wickets lost per ball, etc. Some of the specific challenges were identified:

- **Accounting Non-Legal Deliveries:** Numerous conventional data sets have difficulty with balls staying counts. This study solves the negative balls that is still in error by introducing a new column that adds number of deliveries bowled to date and the counter only decreases on legitimate balls and attains zero at exactly the

end of the 20th over.

- **Data Integrity and Model Protection:** To preserve the integrity of the target variable and prevent any overfitting or noisy data, curtailed, tied, rewarded or ended in a no-result match were dropped.

- **Dataset Coverage:** A major limitation in coverage of the dataset is that the Cricsheet dataset excludes Afghanistan matches [2].

- **Structural Match Identification:** A special Match ID system was necessary to ensure that a duplicate record was not created within the same venue to give each delivery an appropriate context.

- **Target Leakage:** It was a key concern that models should learn generalizable patterns with each target, instead of memorizing class-specific results particularly in the case of the columns herein This was done by creating 2 categories based on whether the team won by runs or by wickets. To prevent this leakage, only the emphasis was maintained on team chasing and the ball-by-ball estimate strictly.

1.3 AIM AND OBJECTIVES

The ultimate aim of the study is to build and experiment with a replicable pipeline that generates approximations of per-ball win probabilities of the chasing team in Men's T20 World Cup matches, which are calibrated.

The objectives of the targeted outcomes are to:

1. Create a reproducible data pipeline to process raw Cricsheet YAML files and convert them into a structured ball-by-ball CSV format.

2. Conduct adequate data cleaning and transformation pre-processing of the dataset to perform venue standardisation and match metadata validation of almost 20 years of T20 matches.

3. Find the right balance between interpretability and accuracy by using various models and comparing them in three classes (Statistical, Machine Learning, and Deep

Learning).

4. Assess model with a 60:20:20 (train, test, validate) split and gain metrics for each set of the data and for each model.

5. Develop a flexible flex code pipeline to enable the addition of future data of the 2026 T20 World Cup without necessarily redesigning the system.

1.4 SCOPE

This dissertation is limited to the completed Men T20 International World Cup matches that will take place between 2007 and 2024. This period was chosen since this is the overall history of the Men T20 World Cup since its inception to the last completed event that was available at the time of data collection.

The matches with a definite result are restricted to analyze as the target variable, win probability of the chasing team, is well defined and consistent throughout the dataset. The tied, abandoned, truncated, or no-result matches were excluded due to the fact that they add ambiguity to the outcome format and diminishes the accuracy of ball-by-ball probability estimation. Another limitation of the study is that it only looks at the second innings, as the goal is to simulate the chances of a successful chase as the game progresses.

Another limitation of the dissertation is that it is based on match-state variables, not on player-specific performance histories. It implies that the models are constructed with objective information about the delivery level (e.g., a run scored, a wicket lost, the number of overs left, and the number of required run rate), as opposed to the reputation of a particular player, or his/her career. This option maintains the structure as generic, replicable and less susceptible to missing player level information.

The dissertation will construct a narrow and comparative win probability model that indicates the actual stress and the team tactics of a Men's T20 World Cup chase by constraining the study to one format of international tournament and a set of completed matches that are free of any interference.

2. LITERATURE REVIEW

Research on sports prediction shows a clear move from older, intuition based methods to newer data driven approaches. In cricket, some early studies used dynamic programming to estimate scoring rates and win probability in one day matches [3]. Bayesian approaches have also been applied to football tournaments to estimate win, draw, and loss probabilities at different stages of a match [4]. Conformal prediction methods have been used to produce well-calibrated win probabilities in sports tournaments such as NCAA basketball [5]. In another example, studies on the English Premier League showed that feature engineering and gradient boosting could produce predictions close to bookmaker accuracy [6].

As data quality improved, researchers began focusing more on live match forecasting. Akhtar and Scarf developed multinomial logistic regression models for Test cricket and found that pre match factors matter more in the early part of the game, while in play signals become more important later [7]. Generalized non-linear models have been used to predict second innings totals and win margins in T20 Internationals [8]. Later this was extended with a dynamic logistic regression model for ODI win probability, where the parameters change gradually as the match goes on [9]. Bayesian dynamic logistic regression models have been applied to IPL matches, with techniques such as Gaussian processes and Brier scores used to improve live win prediction [10]. Other studies also showed that different variables can affect different teams in different ways [11]. Match prediction models have been further improved by incorporating player quality and team form as additional features [12].

Machine learning has also become important in cricket prediction. Methods such as Naive Bayes have been shown to outperform simple run-rate based approaches when using 5-over interval data [13]. Since 2020, many studies have compared models such as Random Forest, Decision Trees, SVM's, K Nearest Neighbors, Naive Bayes, and XGBoost across ODI and T20 matches [14, 15, 16, 17, 18, 19, 20].For T20 cricket, Random Forest and XGBoost have often been used for match winner prediction and score estimation [21,

22]. These models are also commonly combined with features like venue, toss decision, and team strength [23, 24, 25].

Some of these methods have been turned into real time tools and web applications [26, 27]. Even so, many studies that use models like Random Forest and Multi Layer Perceptrons still struggle with probability calibration [28, 29, 30]. In many cases, they do not perform better than simpler regression models when the goal is reliable win forecasting. This shows that high classification accuracy does not always mean better probabilistic prediction. To improve results, some researchers have started adding more detailed player level features, matchup information, and bowler economy rates [31, 32, 33], but this often makes the model more complex and harder to generalize.

Deep learning tries to capture match momentum more directly. Long Short-Term Memory (LSTM) networks have been used to capture time-based patterns in matches and have shown improved performance over standard baseline models [34]. LSTM models, particularly when combined with attention mechanisms, have been shown to capture more long-term dependencies that traditional machine learning models often tend to miss [35, 36]. Similar event level studies in Major League Baseball also show that data driven decisions can sometimes perform better than managerial intuition in fast changing situations [37].

2.1 RESEARCH GAPS AND DISSERTATION CONTRIBUTION

While the existing literature provides a really good foundation for sports forecasting, several critical gaps remain.

First, a significant portion of the current research focuses either on One Day Internationals [23, 9, 12, 38] or franchise-based T20 leagues such as the IPL [24, 10, 27, 20]. Franchise leagues exhibit a high amount of uncertainty and heterogeneity in team compositions due to annual auctions and localized rules. There is a distinct lack of longitudinal studies exclusively targeting the standardized, high-pressure environment of the Men's T20 World Cup spanning its entire history.

Second, several predictive models rely heavily on pre-match variables, player-specific

metrics, and historical team strength indicators [12, 11, 32, 33]. While useful for pre-match forecasting, these variables introduce severe high-dimensionality and subjective biases into live models. There is a need for an analyst-centric approach that isolates purely objective, dynamic match-state variables to evaluate the raw trajectory of a chase without relying on individual player reputations.

Finally, the explosion of machine learning applications in cricket has often been accompanied by critical issues with model calibration [28, 29, 30]. In sports modeling complex algorithms often have problems with target leakage and overfitting. They memorize the training data of understanding the real match dynamics. This leads to predictions that seem confident but are actually wrong.

This dissertation tries to fix these problems in the ways:

- **Continuous Dataset Utilization:** We use a dataset of Men's T20 World Cup matches from 2007, to 2024 aligned from start to end with ball-by-ball data. This helps us focus on a high-pressure situation.

- **Data Engineering:** We create a system to process raw data from YAML files to convert to CSV form and fix errors.

- **Match-State Isolation:** Deliberately isolating pure match-state variables to provide a generalizable framework for real-time win probability estimation without relying on subjective player reputations.

- **Overfitting Mitigation:** We look at the match state to estimate the chance of winning in time. We do not rely on any individual player reputations/performance. Also match venue and toss decision aspect is also not the focus

3. METHODOLOGY

The methodology chapter outlines the systematic technical framework developed to produce calibrated win probabilities for Men's T20 World Cup matches. The approach is divided into phases: collection of the raw data, pre-processing the data, exploratory analysis, feature engineering, various category development of models and final model evaluation.

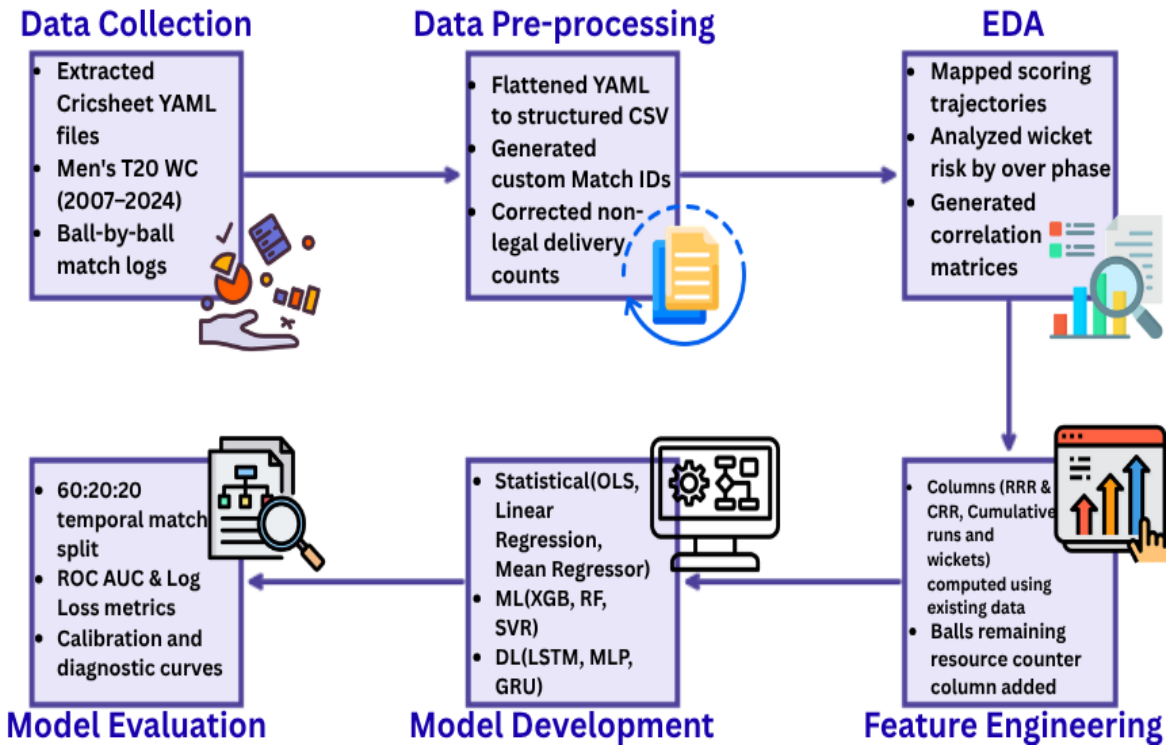

Figure 3.1: Methodological workflow for win probability estimation

3.1 DATA COLLECTION

The data for this research was mainly obtained from the Cricsheet repository [1] on 31st July 2025, the data is mainly available in either of 2 forms YAML or JSON, for this research it was more suitable to work with YAML so it was chosen accordingly. The website provided ball by ball records for international as well as domestic cricket matches, but for this study the focus is mainly on Mens T20 International matches.

Cricsheet data is generated through the algorithmic parsing of public domain match commentary and scorecards. This transformation of the data plays a critical role for establishing the main analytical aspect of this research:

- **Entity Mapping:** Mapping names to unique alhabet/numeric registry IDs and values from data.
- **Event Categorization:** Parsing descriptions into keys such as `batsman_runs` and `wicket_kind`.
- **Temporal Sequencing:** Assigning a precise index to every ball.

Figure 3.2: Raw YAML file showing ball-by-ball info [1]

3.2 DATA PRE-PROCESSING

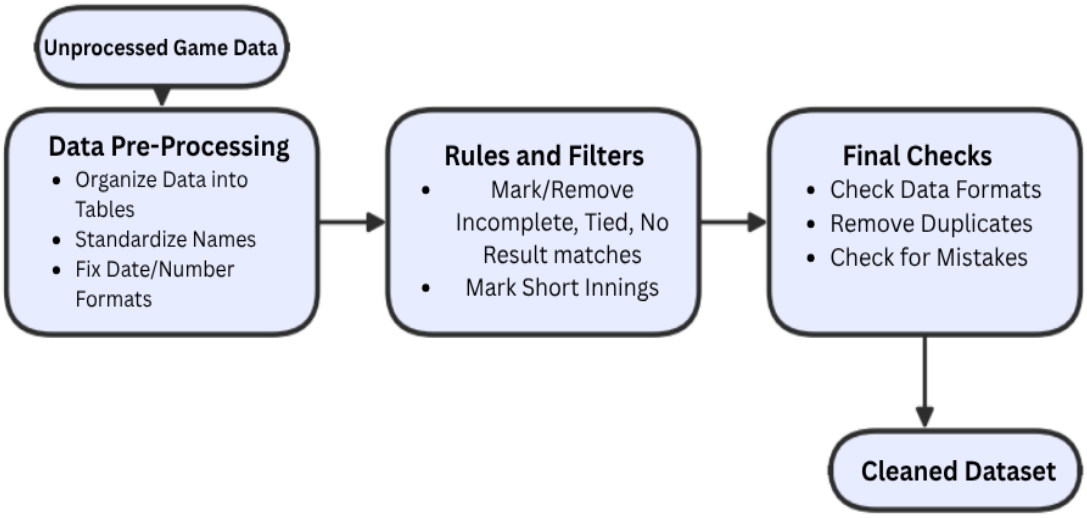

Figure 3.3: Logic flow for data preparation, standardization, and match filtering

Using Python code, each individual YAML file was processed to make sure that only T20 World Cup matches are included in converted dataset, further we remove the curtailed and no- result matches as well. Then nested hierarchy from each individual YAML files was flattened into a ball by ball CSV format and combined into a single dataset where every row represents a single delivery. As shown in Figure 3.4, the deliveries are aligned chronologically starting from 2007, arranged first by matchid then inning number, over number, ball number. This results in systematic alignment of data starting from exact first ball bowled in 2007 until the latest ball bowled in the final of 2024 world cup. Using the converted dataset we first create the engineered features and columns that will be needed later on as show in below Table 3.1.

	A	B	C	D	E	F	G	H	I	J
1		match_id	date	venue	city	country	innings_number	batting_team	bowling_team	over
2	0	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
3	1	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
4	2	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
5	3	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
6	4	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
7	5	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	1
8	6	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
9	7	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
10	8	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
11	9	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
12	10	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
13	11	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	2
14	12	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
15	13	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
16	14	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
17	15	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
18	16	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
19	17	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
20	18	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	3
21	19	20070911_NewWand	2007-09-11	New Wanderers Stad	Johannesburg	South Africa	1	West Indies	South Africa	4

Figure 3.4: Structured CSV representation of ball-by-ball match data

3.2.1 Data Dictionary and Feature Definitions

To ensure technical transparency and reproducibility, Table 3.1 provides a comprehensive data dictionary of the attributes present in the final processed dataset. These features, extracted and engineered from the raw Cricsheet data, are categorized into match metadata, ball by ball delivery information, and engineered temporal proxies.

Table 3.1: Dataset columns and definitions

Column Name	Data Type	Description
Group 1: Match Metadata		
match_id	String	Custom unique identifier (e.g., 20070911_Venue_Teams).
date	Date	Calendar date of the match.
venue / city / country	String	Geographic location and stadium name.
toss_winner	String	The team that wins the coin toss(50/50)
toss_decision	String	Toss winner decision to bat or field first.
winner	String	The final winning team.
win_type / margin	Mixed	Method and numerical value of victory.
player_of_match	String	Player awarded the MVP for the match.
Group 2: Ball by Ball Delivery Information		
innings_number	Integer	1 for batting first; 2 for chasing.
batting / bowling_team	String	Active teams for the current delivery.
over / ball	Float	Precise delivery index (e.g., Over 1, Ball 3).
batsman / non_striker	String	Identity of the two active batters.
bowler	String	Player delivering the ball.
runs_batsman / extras	Integer	Runs scored off the bat vs penalty runs.
runs_total	Integer	Cumulative runs for the delivery.
extras_type	Category	Classification of extras (e.g., wide, no ball).
wicket_type	Category	Method of dismissal (e.g., caught, bowled).
player_dismissed	String	Name of the player who lost their wicket.
fielder	String	Fielder involved in the dismissal.
Group 3: Engineered Features and Temporal Proxies		
is_boundary / dot_ball	Boolean	Indicator for 4s or 6s or zero run deliveries.
cumulative_runs	Integer	Total runs scored in the innings up to that ball.
cumulative_wickets	Integer	Total wickets lost in the innings up to that ball.
run_rate	Float	The Current Run Rate (CRR).
required_runs	Integer	Runs remaining to reach the target.
balls_remaining	Integer	Legal deliveries left in the 20 over quota.
required_run_rate	Float	Target runs divided by remaining legal overs.
over_phase	Category	Match phase: powerplay, middle, or death overs.

3.3 EXPLORATORY DATA ANALYSIS (EDA)

3.3.1 Dataset Summary and Meta-Statistics

Before performing delivery level analysis of the data, a statistical summary of the filtered Cricsheet data was done. This dataset [1] contains almost 2 decades of T20 matches full of elite teams, players playing at international level and for their country. To make sure that integirty of win probability for chasing team gives accurate outputs, the analysis is strictly limited to completed matches, not including matches that were tied, no-result, curtailed, etc [2].

Table 3.2: General Dataset Meta-Statistics (2007–2024)

Attribute	**Value / Details**
Total Match Editions	9 Tournaments
Temporal Range	2007 – 2024
Total Completed Matches Analyzed	259
Total Ball-by-Ball Deliveries	60,931
Geographic Spread	42 Venues across 40 Cities in 11 Countries
Total Participating Teams	23
Total Unique Players	932
Win Distribution (Batting First)	49.03%
Win Distribution (Chasing)	50.97%
Top Run Scorer	V Kohli (1,218 runs)
Overall Average Runs (per Batter per Match)	17.00
Top Wicket Taker	Shakib Al Hasan (47 wickets)
Overall Average Wickets (per Bowler per Match)	1.66
Top Catchers (Fielding)	AB de Villiers (27), GJ Maxwell (21)
Primary Dismissal Types	Caught (1940), Bowled (642), Run Out (251)

To better understand the dynamics of the T20 format, the match features were broken down into comparisons between each inning. Table 3.3 highlights the variance in scoring, wicket loss, and boundary frequency between the teams batting first and the teams chasing. This was done using all the matches from the dataset to get a good idea of the

statistics.

Table 3.3: Innings-wise Feature Distributions and Boundary Statistics

Match Feature	**Innings**	**Minimum**	**Average (Mean)**	**Maximum**	**Global Total**
Final Score	1st Innings	40	152.16	260	74,142
	2nd Innings	39	134.10	230	
Wickets Fallen	1st Innings	2	6.69	10	3,283
	2nd Innings	0	5.98	10	
Fours Hit	1st Innings	3	12.16	30	5,973
	2nd Innings	2	10.90	24	
Sixes Hit	1st Innings	0	4.81	16	2,371
	2nd Innings	0	4.35	19	
Dot Balls	1st Innings	25	44.83	87	22,681
	2nd Innings	10	42.74	78	

The above match meta-statistics reveal the base characteristics of the T20 World Cup environment as a whole. By using completed matches, the dataset provides a clean, isolated variables to be used for win probability estimation. Before modeling started, EDA was performed to identify the primary drivers of match outcome and get a good idea. This next phase mainly focuses on quantifying how scoring rates, wickets fallen risk changes across the three different match phases:

- **Powerplay:** Ranges from overs 1-6. This acts as the opening phase of both innings, in which the fielding restrictions are high and only 2 fielders are allowed to stand near the rope, this allows for opening batters to be much more aggressive and hit the ball over the fielders who are mostly placed near the batting pitch. This phase

is crucial for batting side as high number of runs will result in less pressure in later phases.

- **Middle Overs:** Ranges from overs 7-15. Once powerplay ends, the fielding restriction is lifted and it's much more difficult to score boundaries unless batter aims to hit direct six into the stands or to perfectly time and hit the ball inbetween the gaps in the field. Run rate tends to slow down here as a result and main objective is to save wickets, play bit slow until final death overs.

- **Death Overs:** Ranges from overs 16-20. Once the inning reaches last few overs, if the batters have managed to save their wickets by playing much more in defensive form, then they are allowed to play with lot more aggression as they can afford to lose wickets and let the lower order batter finish the job, therefore run rate skyrockets in this phase, especially when chasing team gets closer to target and wishes to not take the chase till final over and finish before 20th over itself.

3.3.2 Correlation Analysis

A correlation matrix was plotted using the features from the dataset. As expected, "required_run_rate" and "cumulative_wickets" show the strongest negative correlation with chasing success. While cumulative runs and run rate showed high correlation, this shows how the chasing team requires some key columns to identify and compute the win probability at any point in time during the match.

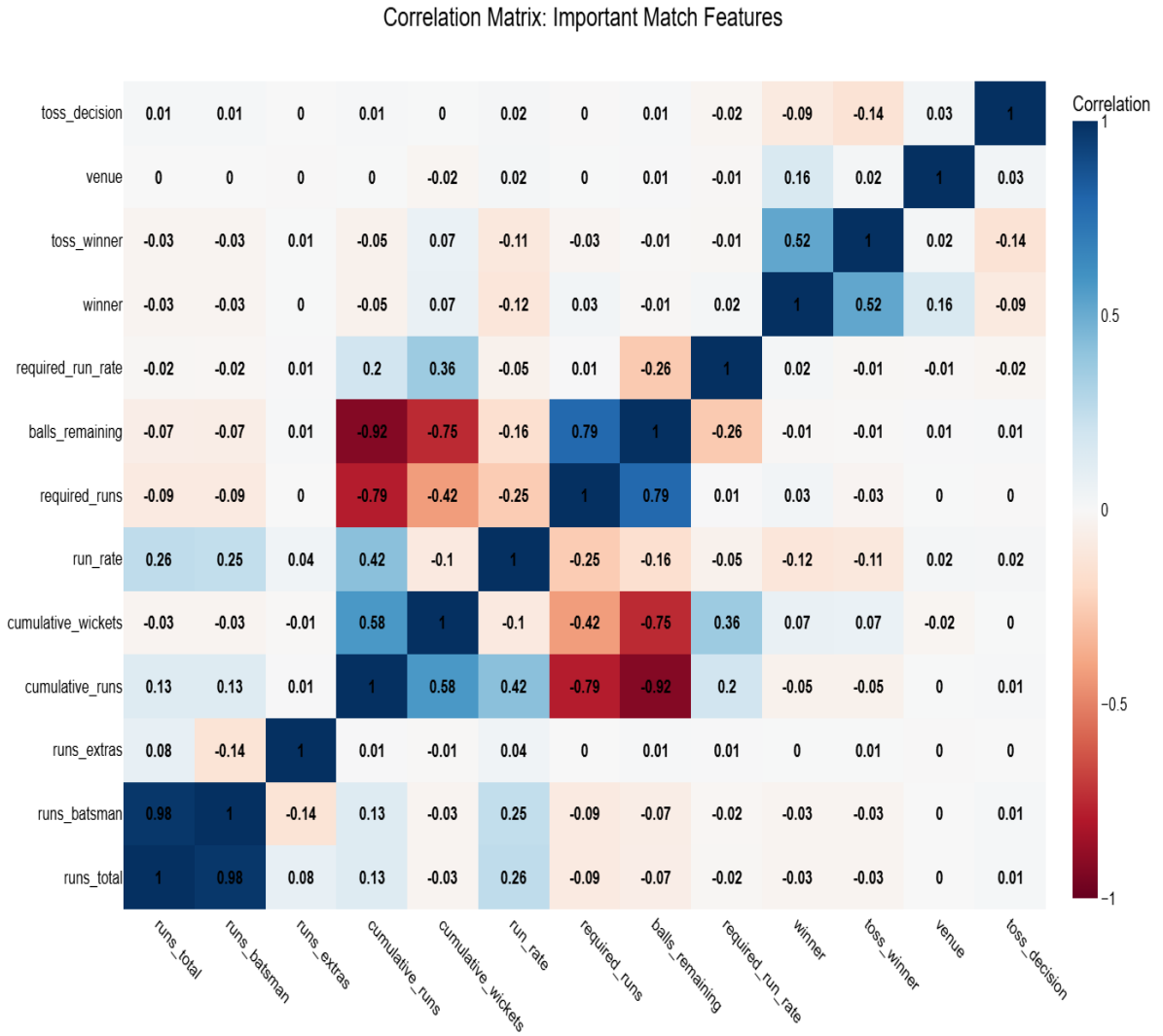

Figure 3.5: Correlation matrix of match state features

3.3.3 Scoring Patterns, Boundary Aggression, and Wicket Risk Across Innings

To understand how a T20 chase develops, scoring rate, boundary frequency, and wicket loss were studied together across both innings. These three views describe the same match from different angles. They show how quickly runs are being scored, how often attacking shots are played, and how the risk of losing wickets changes as the innings

moves forward.

Taken together, these patterns show that the chase becomes more pressure driven as the match moves from the Powerplay phase to the Middle Overs and then to the Death Overs.

The scoring pattern in Figure 3.6 shows a clear phase based trend. Runs per over are usually higher in the Powerplay because of fielding restrictions. They then settle during the Middle Overs as teams try to build their innings. In the Death Overs, the scoring rate rises again as teams push for more runs in the final overs. The chasing innings is more uneven than the first innings because the required run rate keeps changing. In many failed chases, the scoring rate also drops late in the innings. This makes the second innings more unpredictable and more important for win probability estimation.

Boundary frequency in Figure 3.7 shows a similar pattern. Boundaries are more common in the Powerplay, then drop during the Middle Overs, and rise again in the Death Overs when teams take more risks. This shows that scoring aggression is not the same throughout the innings. It changes with the match situation. Late boundary bursts often decide whether a chase stays alive or falls apart.

The wicket pattern in Figure 3.8 adds another important layer. Wickets are fairly balanced in the early overs, but they rise in the Death Overs, mainly in the second innings where the chasing side has to attack while still trying to survive. This is one of the most important signals in the project because a chase is not only about scoring quickly. It is also about keeping wickets in hand long enough to reach the target. The rise in wicket loss near the end explains why cumulative wickets, wickets per over, and required run rate are such important features in the prediction model.

Overall, these three analyses show that a T20 chase is shaped by the balancing between

scoring fast, balls remaining and wicket preservation, and that balance becomes much more intense in the final overs.

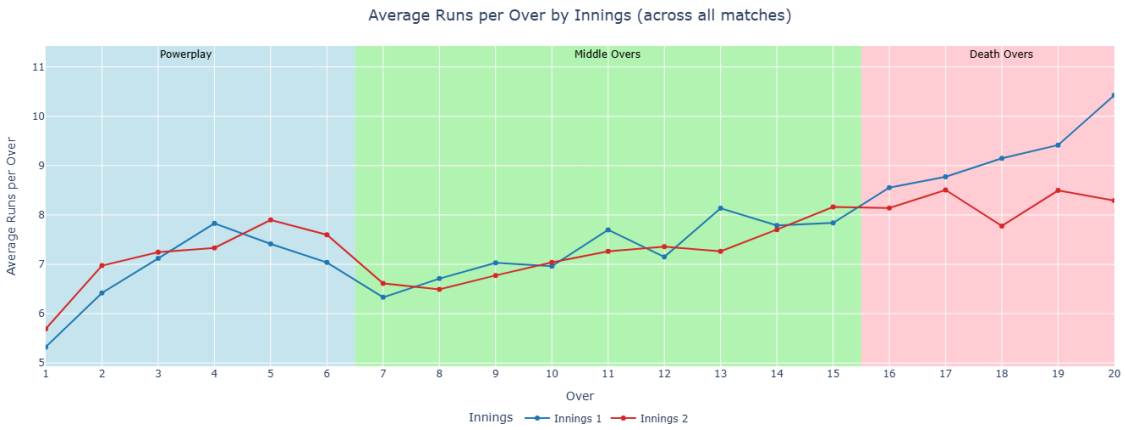

Figure 3.6: Average runs per over by innings

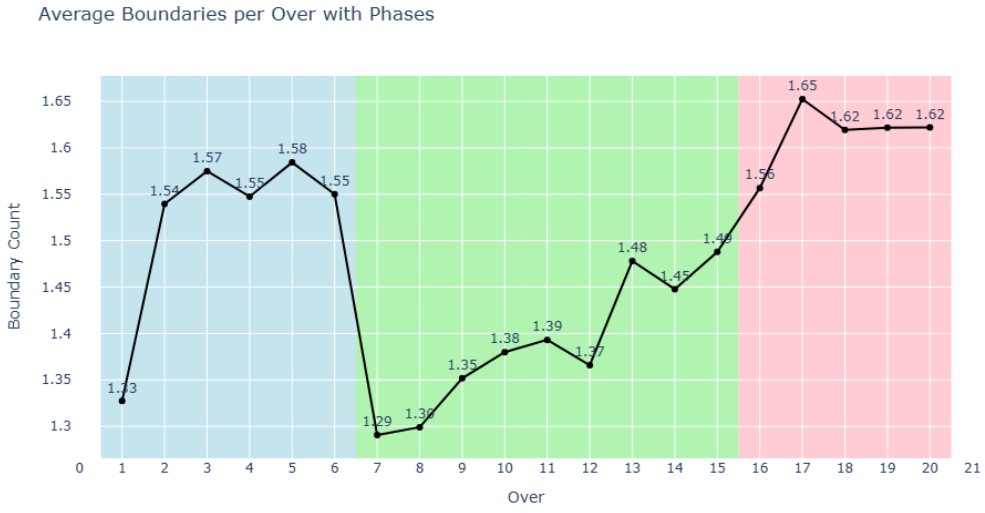

Figure 3.7: Average boundaries per over across match phases

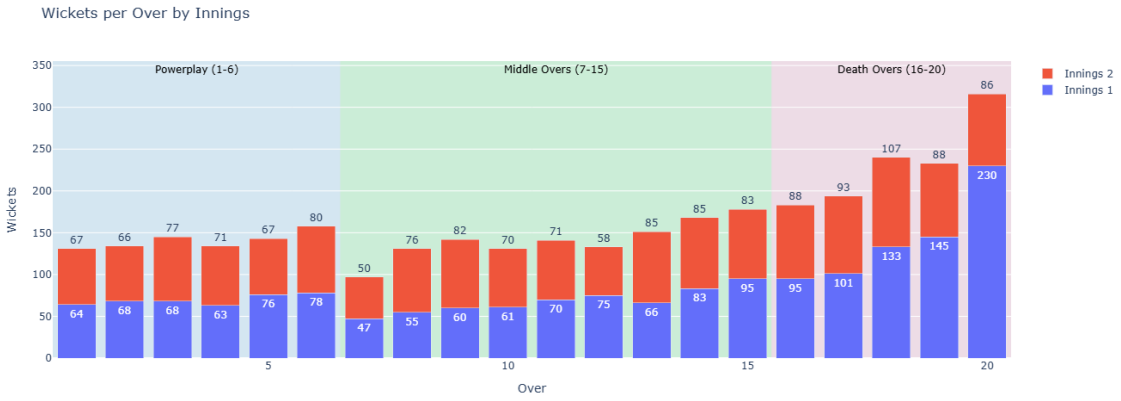

Figure 3.8: Wickets per over by innings

3.4 FEATURE ENGINEERING

Feature engineering acts as the bridge between the transformed dataset and the models which will be implemented later on. While basic features such as runs and wickets help a little bit for prediction, they aren't the best suited for capturing the buildup of pressure during chase in a T20 match. Various features were created using exisitng features and to better gain a better perspective of the dataset.

3.4.1 Dynamic Rate Metrics and Resource Calculation

The most important features that will be used for the models are engineered using exisitng features. First, is Current Run Rate (CRR) and second is Required Run Rate (RRR). Unlike normal static match averages, these engineered features were calculated for each and every single delivery in the dataset

Current Run Rate represents the scoring rate of the batting team, calculated as the runs scored per over uptil point of calculation and is given by:

$$CRR = \frac{\text{Cumulative Runs}}{\left(\frac{\text{Legal Deliveries Bowled}}{6}\right)} \tag{1}$$

While, the Required Run Rate represents the exact scoring rate that is required for the chasing team and is only available for innings 2. The RRR was calculated by subtracting cumulative runs at that delivery from target score for that particular match divided by the remaining number of balls(legal deliveries) in standard over form. This feature is important as it quantifies the remaining amount of effort is needed for the chasing team to win and it's given by:

$$RRR = \frac{\text{Target Score} - \text{Cumulative Runs}}{\left(\frac{\text{Balls Remaining}}{6}\right)} \tag{2}$$

Additionally, a "Balls Remaining" counter was used in order to make sure legal deliveries were properly counted and 20 over quota was properly filled. This allowed for better tracking of overs without having illegal deliveries in the mix. By using these dynamic metrics, the models can effectively learn the underlying trade-offs between the features at any specific match state.

3.5 MODEL DEVELOPMENT

This research implements 3 separate category of models including statistical models(OLS, Linear Regression, Mean Regressor), machine learning models(Random forest, XGBoost, SVR) and deep learning models(MLP, LSTM, GRU).

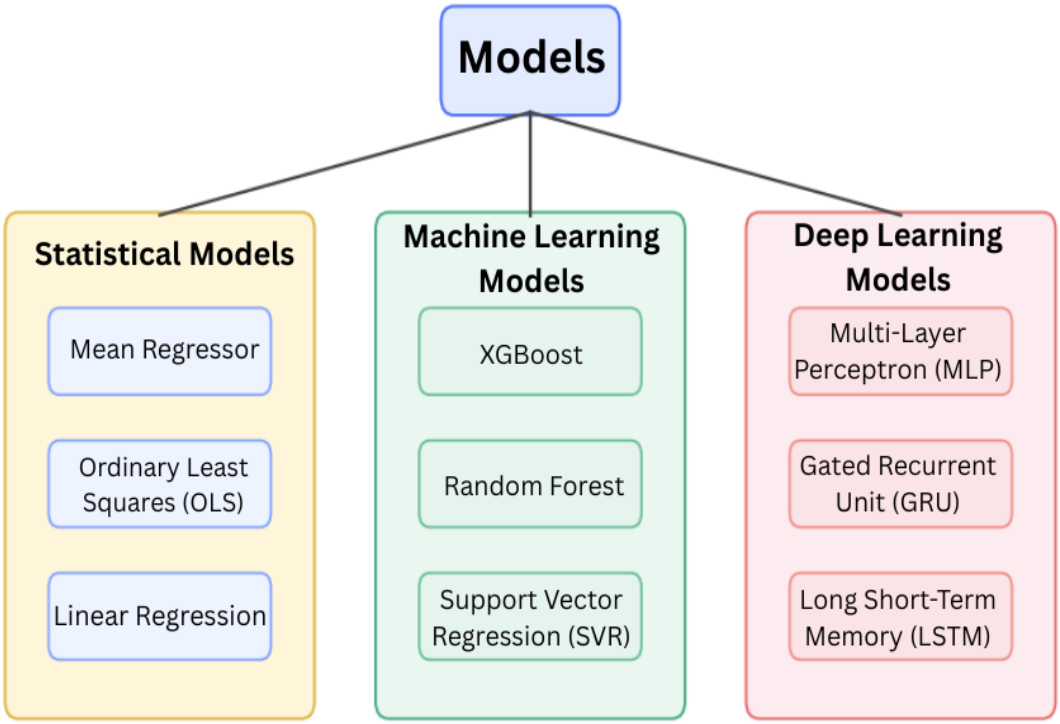

Figure 3.9: Hierarchical representation of models

3.5.1 Statistical Models

The statistical models were included first because they give a clear and easy to interpret starting point for win probability estimation. In statistical learning, they help check whether the engineered match state variables already contain enough useful ability before moving to more flexible models [39]. In this dissertation, these models are used to see how far a mostly linear relationship between chase pressure and match outcome can go. If they perform well, it suggests that variables such as required run rate, cumulative runs, wickets lost, and balls remaining already explain a large part of the chase.

a. Mean Regressor: The Mean Regressor predicts the average target value from the training data for every case. It does not use any match features, so it works as a basic

baseline with no real information. This model is included to show the lowest level of performance that any useful win probability model can easily beat. In this project, it also helps show whether the cricket based engineered features add value beyond a simple constant prediction [39].

b. Ordinary Least Squares (OLS): Ordinary Least Squares, estimates the relationship between the target and the input variables by minimizing the total squared error. It is widely used because it is simple, interpretable, and mathematically well established [40]. In this study, OLS is used to measure how well a linear combination of match state variables can explain the chasing team's chance of winning. It also gives a useful reference point for comparing more advanced models.

c. Linear Regression: Linear Regression is used as another linear benchmark to check whether the pattern seen in OLS is stable or not [41]. Similar to OLS, it models the target as a weighted combination of input features. In this dissertation, it helps test whether the chase dynamics in Men's T20 World Cup matches can be explained through a simple linear pressure relationship. If its results are close to OLS, that supports the idea that the main signal in the data is mostly linear.

3.5.2 Machine Learning Models:

ML models are the next category as they can capture non-linear relationships and feature interactions that linear models may not be able to capture. Cricket chases are not driven by one variable at a time. The effect of a wicket, for example, depends on how many balls remain and how large the target is. Models in this category are therefore useful for learning threshold effects and more complex decision boundaries. Compared

with the statistical models, they are more expressive, and compared with deep learning, they remain more manageable on structured tabular data.

a. XGBoost: XGBoost is a scalable tree boosting system that builds extra decision trees in a straight sequential way, wherein each new tree improves the errors made by the preceding trees [42]. It is especially effective for structured tabular data and is able to capture non-linear patterns, interaction effects, and irregular decision boundaries. In this dissertation, XGBoost is used to model complex chase behavior in phases where the relationship between score pressure and win probability changes sharply, especially in the middle and death overs.

b. Random Forest: Random Forest is a type of ensemble method which constructs many decision trees on bootstrapped samples and averages their predictions to reduce variance [43]. This model is a good match for the research as cricket outcomes often depend on various combinations of the conditions rather an individual static factor. It is also used to capture sudden probability changes when match state changes dynamically, such as after a wicket falls or an expansive over for the bowler.

c. Support Vector Regression (SVR): Support Vector Regression improves the support vector framework to the regression tasks by fitting a function within an epsilon-insensitive margin while minimizing prediction error [44]. With the radial basis function kernel, SVR can model curved and non-linear relationships in the feature space. It is included in this project because the effect of variables such as wickets in hand, required runs, and remaining overs is often not proportional. SVR is therefore a strong candidate for learning the smooth but non-linear structure of a chase.

3.5.3 Deep Learning Models:

The deep learning models form the final category because they are designed to learn sequential and temporal structure directly from the innings trajectory. Ball-by-ball win probability is not only a feature to outcome problem, but also a sequence problem, since the state of the chase depends on what happened in the previous deliveries. Deep learning is therefore useful for modeling momentum, pressure accumulation, and the evolving context of the innings. The neural models in this study build on classic ideas in representation learning and recurrent sequence learning [45, 46, 47].

a. Multi Layer Perceptron (MLP): The Multi Layer Perceptron is a feed-forward neural network that learns non linear combinations of input features through stacked dense layers and back-propagation [45]. In this dissertation, the MLP is used to test whether a non-sequential neural architecture can improve on classical machine learning models when provided with engineered match-state features. Its role is important because it separates the effect of neural non-linearity from the effect of temporal memory.

b. Gated Recurrent Unit (GRU): The Gated Recurrent Unit or GRU is a type of recurrent neural network. It uses update as well as reset gates to control how information moves through a sequence [46]. This makes it useful for learning from ball by ball data while filtering out less important noise. In this research, we use GRU because a chase develops gradually. The most recent deliveries often have the biggest impact on the next ball. The model is expected to pick up on momentum shifts and pressure changes more naturally than older methods that do not account for sequences.

c. Long Short Term Memory (LSTM): Long Short Term Memory or LSTM is a type of recurrent architecture with a memory cell and gating system. This helps it keep track of long range dependencies across a sequence [47]. It is included in this dissertation because early balls in a chase can still affect the final result. This is especially true when a team builds or loses momentum over several overs. LSTM learns this longer context and creates smoother probability results for the chasing team. Of the deep learning models used, it is the best at capturing the steady rhythm of a match across a full innings.

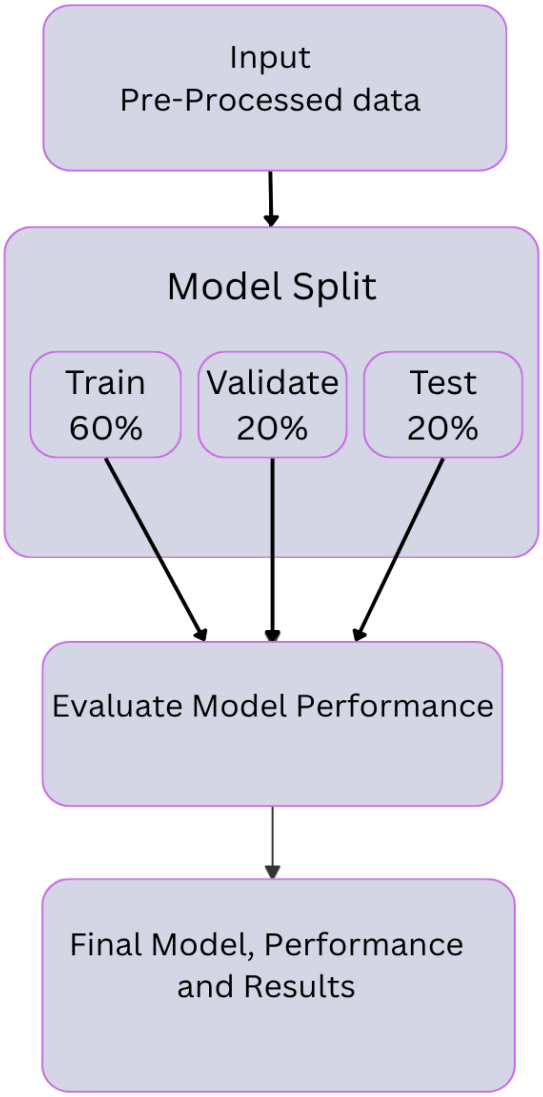

Figure 3.10: Model Evaluation Pipeline Utilizing a 60-20-20 Data Split

3.5.4 Hyperparameter Configuration

All the models in this research were kept simple and uniform to ensure a fair comparison. A fixed seed of 42 was used across the pipeline, which made data splits and training results easy to reproduce whenever needed. A random seed is a starting point for the number generator. Since these systems typically follow set rules, using the same seed every time makes sure that the output remains the same and consistent when code is ran multiple times.

The statistical models didn't need much adjustment. Mean Regressor was used as simple starting point by looking at average training target. OLS and Linear Regression used a standard linear formula. This helped the modelsmap out general connections between the match data and the win probability.

For the ML model, the ensemble models were set to 200 trees. This was enough to capture complex patterns without having the training become too slow. Also the features were scaled for the SVR model. This is very important as SVR works much better when all inputs values are on a similar scale.

For the DL models, Adam optimizer was used with a specific learning rate, with a batch size of 256 to balance the training speed with stability. To prevent models from just memorizing the data, dropout and batch normalization was used.

For GRU and LSTM, the input length was fixed to 200 balls, along with a masking layer so that extra padded values wouldn't interfere with the learning process. At the end, early stopping was used to end the training once the model stopped showing signs of progress on the validation data.

Table 3.4: Simplified hyperparameter settings used in the study

Category	**Model / Parameter**	**Setting used and reason**
Statistical	Mean Regressor	Predicts the average training target, used as a baseline.
Statistical	OLS / Linear Regression	Standard linear setup with constant term, used for interpretable comparison.
Machine Learning	XGBoost / Random Forest	200 estimators, chosen to balance performance and runtime.
Machine Learning	SVR	Standard scaling applied, because SVR is sensitive to feature scale.
Deep Learning	Learning rate	1×10^{-3} with Adam optimizer, used for stable training.
Deep Learning	Batch size	256, chosen as a balance between speed and stability.
Deep Learning	Regularization	Dropout 0.25 and batch normalization, used to reduce overfitting.
Deep Learning	Sequence models	192 hidden units, sequence length 200, masking enabled for GRU/LSTM.
Deep Learning	Early stopping	Patience 10 on validation AUC, used to keep the best model weights.

3.6 MODEL EVALUATION METRICS

This project treats the win probability task mostly as a probability prediction problem. Because of this, the chosen evaluation metrics measure two things. They check the numerical accuracy of the predictions and also how well the models separate winning states from losing ones.

The target variable falls somewhere between the interval from 0 to 1. To evaluate this properly, the study uses several metrics. These include Mean Squared Error, Root Mean Squared Error, Coefficient of Determination, Accuracy, Log Loss, and ROC AUC.

3.6.1 Mean Squared Error (MSE)

Mean Squared Error looks at the average squared difference between the predicted probability and the actual result [48]. In this work, the actual match outcome is either 0 or 1. The model output is a decimal value in between.

A lower score here means the predictions are closer to the true outcome. This metric is useful because it treats the probability as a continuous number. It rewards models that get numerically close to the right answer.

3.6.2 Root Mean Squared Error (RMSE)

Root Mean Squared Error is simply the square root of Mean Squared Error. It expresses the prediction error on the exact same scale as the probability output [49].

This makes it much easier to interpret since the value is not squared. It gives a very clear picture of the average deviation. This is used to compare how far off the predictions are from the actual results across all the different models.

3.6.3 Coefficient of Determination (R^2)

The R^2 metric shows how much of the variance in the target variable is explained by the model [50]. Although it's usually used more commonly in classic regression problems. However, it is still helpful here because the output is treated as a continuous value.

A higher R^2 score means the model is able to capture more of the underlying changes in the match state. This metric is included as a diagnostic tool to compare the overall strength of the different models in this research.

3.6.4 Accuracy

Accuracy measures the ability of the model to estimate the right outcome for every single delivery [51]. It is included as a basic decision level metric. It shows how often the model correctly identifies the win probability at each delivery.

However, accuracy alone does not tell the whole story. It does not reflect how well the probabilities are calibrated. Therefore, it is only used as a supporting metric and not the main measure of success.

3.6.5 Log Loss

Log Loss evaluates the overall quality of the predictions. It heavily penalizes confident but wrong predictions compared to uncertain ones [52]. This makes it one of the most crucial metrics for any probability task.

In the context of this project, it is highly relevant, the goal is not just to guess the winner, but also to produce reliable win probability ball by ball. A lower score means the model assigns realistic probabilities to the chasing team throughout the innings.

3.6.6 ROC AUC

ROC AUC measures how well a model ranks positive outcomes over negative ones [53].This makes it a strong tool for comparing models that generate probability outputs.

This metric is especially important for this research. It shows whether the model can truly tell the difference between winning and losing chase situations. A higher score means there is a much stronger separation between the two possible outcomes.

3.6.7 Metric Selection for This Study

These metrics were chosen together to evaluate the models from multiple angles. MSE and RMSE check for numerical closeness. R2 measures explanatory strength. Accuracy provides a basic performance score. Log Loss evaluates probability calibration. Finally, ROC AUC measures ranking ability.

Combined, they offer a very balanced assessment as they show whether a model just guesses the right labels or actually produces trustworthy probability estimates suitable for live cricket analysis.

3.7 TOOLS AND INFRASTRUCTURE

This reserach along with the modeling were conducted within the Google Colab Pro environment using high-performance computing resources. The Primary hardware used was an NVIDIA Tesla T4 GPU with a High-RAM system configuration, which helped with the more memory-intensive training of deep learning models.

3.7.1 System and Hardware Specifications

Following is the full list of hardware and software configuration used for this research displayed in Table 3.5. This allows for full transparency and allows for future researchers to replicate this computational pipeline in similar manner.

Table 3.5: Google Colab Pro Environment Specifications

Component	**Specification**
Operating System	Linux (Release: 6.6.113+, x86_64)
CPU	4 Physical Cores (8 Logical Cores)
System Memory (RAM)	50.99 GB
GPU	NVIDIA Tesla T4
CUDA Version	12.8 (CUDA Available: True)
Python Version	3.12.13
NumPy Version	2.0.2
Pandas Version	2.2.2
Scikit-learn Version	1.6.1
TensorFlow Version	2.19.0

3.7.2 Software Frameworks and Methodologies

The project was built using a small set of core tools and methods:

- Python 3.12 was the core programming language used for all data processing, model building, and evaluation tasks. I used Google Colab Pro as the primary development environment because it supports the GPU acceleration and large memory capacity required for deep learning.

- For data engineering, I used pandas and numpy to handle data cleaning, transformation, and all numerical computations. I relied on statsmodels to build the Ordinary Least Squares regression and conduct easy to understand linear analysis.

- The scikit learn library handled the Linear Regression, Random Forest, and SVR models. It was also used for feature scaling and calculating all evaluation metrics. In order to build and train the neural networks, including the MLP, GRU, and LSTM models, I used TensorFlow and Keras.

- Interactive visualizations like ROC curves, calibration curves, and win probability traces were generated using Plotly.

- The raw data originally came in YAML format from Cricsheet. I built a pipeline to convert these files into structured CSV formats so they could be used for modeling. The final dissertation was written and formatted using LaTeX, with BibTeX handling the reference management.

- To make sure all experiments are repeatable, I set fixed random seeds across NumPy, Python, and TensorFlow.

- Finally, I split the dataset chronologically at the match level into training, validation, and test sets using a 60 to 20 to 20 ratio. This setup provides a solid training base while keeping a separate validation set. This validation set is critical for triggering early stopping and preventing the deep learning models from overfitting.

4. ANALYSIS AND CONCLUSIONS

4.1 RESULTS

Table 4.1: Performance metrics for all models across training, validation, and test sets

(a) Training and Validation Sets

	MSE		**RMSE**		R^2		**Acc.**		**Brier**		**Log Loss**		**ROC AUC**	
Model	**Tr***	**Val***	**Tr**	**Val**	**Tr**	**Val**	**Tr**	**Val**	**Tr**	**Val**	**Tr**	**Val**	**Tr**	**Val**
Mean Regressor	0.2484	0.2533	0.4984	0.5033	0.0000	-0.0150	0.5394	0.4781	0.2484	0.2533	0.6900	0.6997	0.5000	0.5000
OLS	0.1457	0.1247	0.3817	0.3531	0.4137	0.5004	0.8031	0.8292	0.1457	0.1247	0.4611	0.3857	0.8821	0.9312
Linear Regression	0.1457	0.1247	0.3817	0.3531	0.4137	0.5004	0.8031	0.8292	0.1457	0.1247	0.4611	0.3857	0.8821	0.9312
XGBoost	0.0016	0.1645	0.0402	0.4056	0.9935	0.3409	0.9996	0.7716	0.0016	0.1645	0.0224	0.8338	1.0000	0.8519
Random Forest	0.0184	0.1415	0.1355	0.3762	0.9261	0.4327	0.9996	0.7930	0.0184	0.1415	0.1073	0.4499	1.0000	0.8921
SVR	0.1228	0.1289	0.3505	0.3590	0.5056	0.4835	0.8277	0.8196	0.1228	0.1289	0.5107	0.5723	0.9034	0.9134
MLP	0.1719	0.1586	0.4146	0.3983	0.3080	0.3643	0.7897	0.8250	0.1719	0.1586	0.5236	0.4935	0.8865	0.9155
GRU	0.1350	0.1246	0.3674	0.3530	0.4567	0.5006	0.7911	0.8250	0.1350	0.1246	0.4062	0.3801	0.8991	0.9082
LSTM	0.1396	0.1227	0.3736	0.3503	0.4382	0.5083	0.7869	0.8207	0.1396	0.1227	0.4173	0.3731	0.9000	0.9110

* *Tr = Training Set, Val = Validation Set*

(b) Test Set

Model	**MSE**	**RMSE**	R^2	**Acc.**	**Brier**	**Log Loss**	**ROC AUC**
Mean Regressor	0.2511	0.5011	-0.0045	0.5059	0.2511	0.6953	0.5000
OLS	0.1443	0.3798	0.4229	0.8118	0.1443	0.4395	0.8870
Linear Regression	0.1443	0.3798	0.4229	0.8118	0.1443	0.4395	0.8870
XGBoost	0.1701	0.4124	0.3195	0.7668	0.1701	0.9283	0.8388
Random Forest	0.1555	0.3944	0.3778	0.7850	0.1555	0.4874	0.8548
SVR	0.1468	0.3831	0.4128	0.7936	0.1468	0.5154	0.8728
MLP	0.1786	0.4226	0.2856	0.7701	0.1786	0.5386	0.8638
GRU	0.1666	0.4081	0.3337	0.7519	0.1666	0.5020	0.8479
LSTM	0.1749	0.4182	0.3002	0.7241	0.1749	0.5141	0.8405

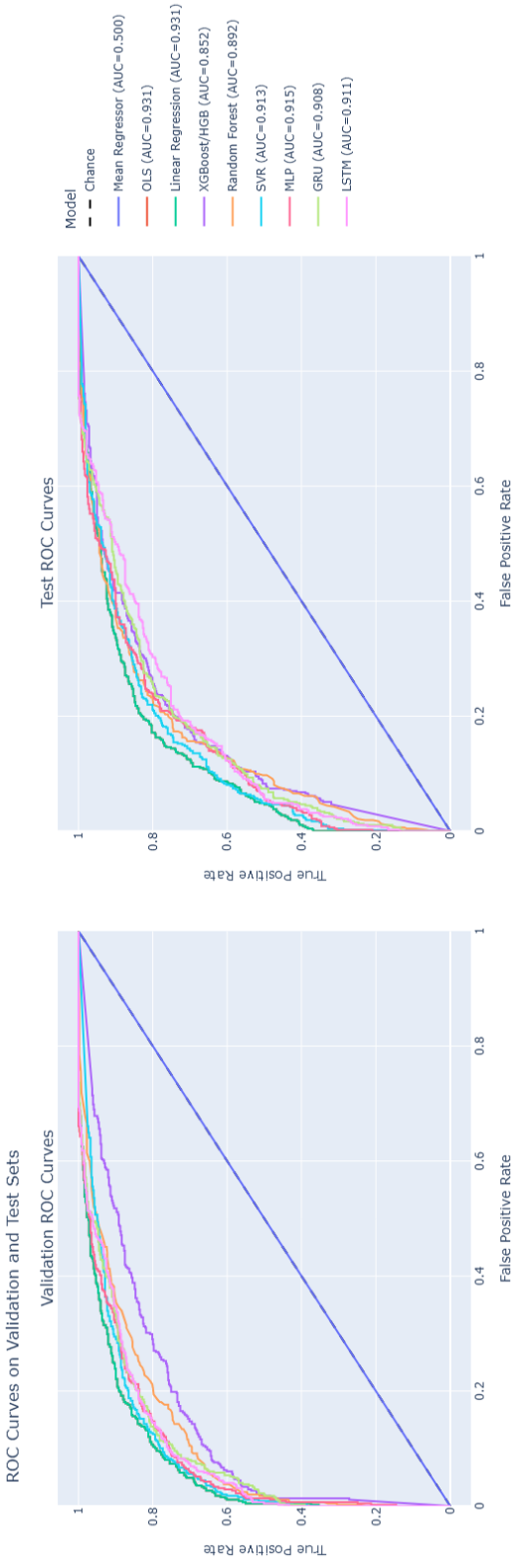

Figure 4.1: ROC curves on validation and test sets for all models

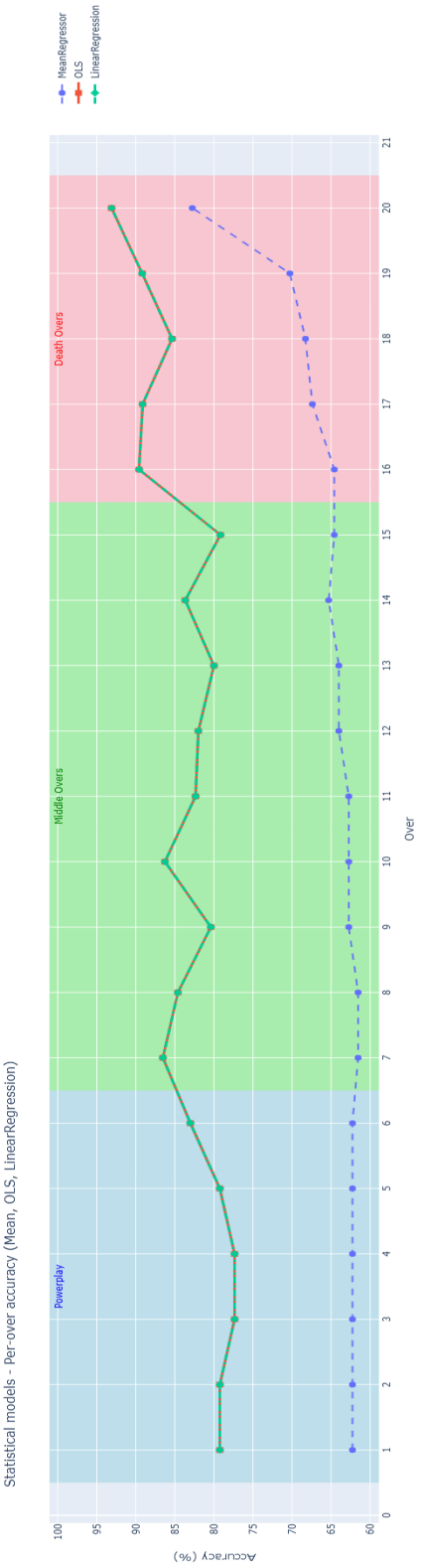

Figure 4.2: Per-over accuracy for statistical models

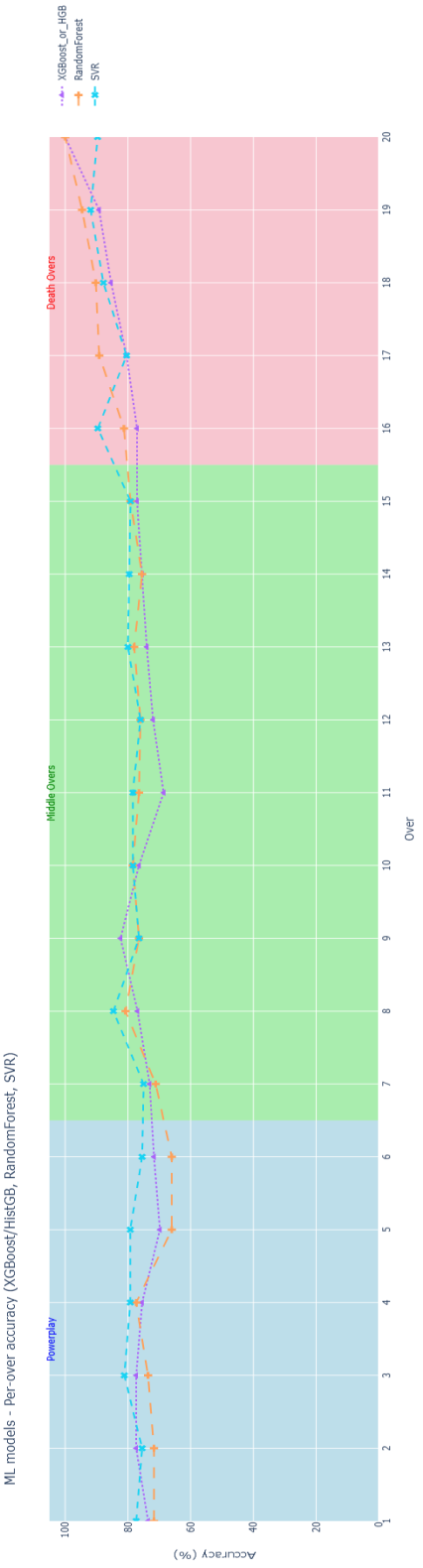

Figure 4.3: Per-over accuracy for machine learning modelst

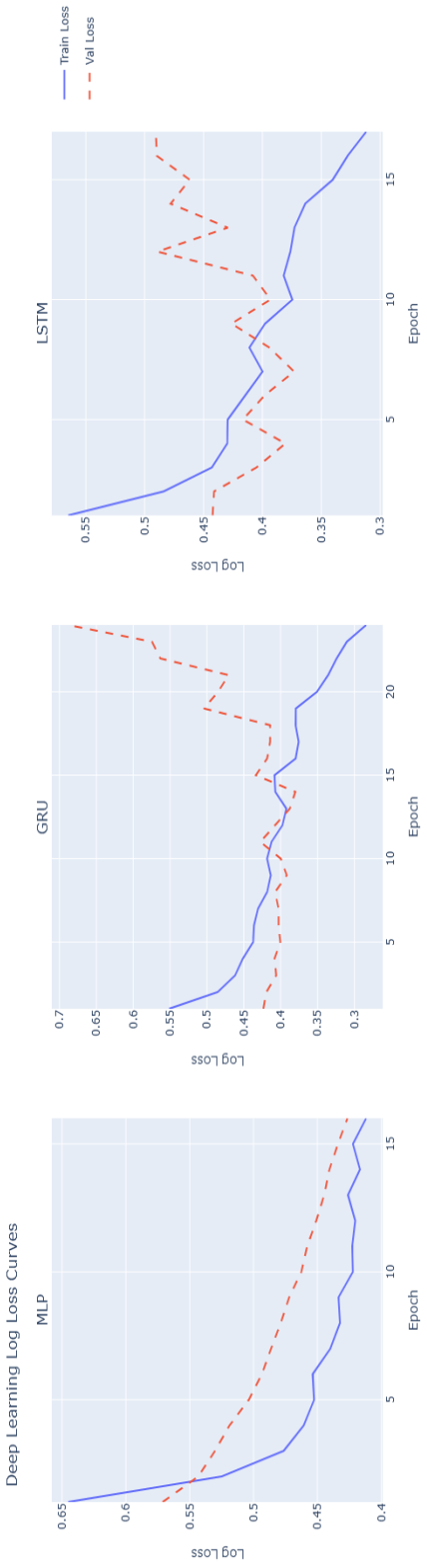

Figure 4.4: Training and validation log loss curves for deep learning models

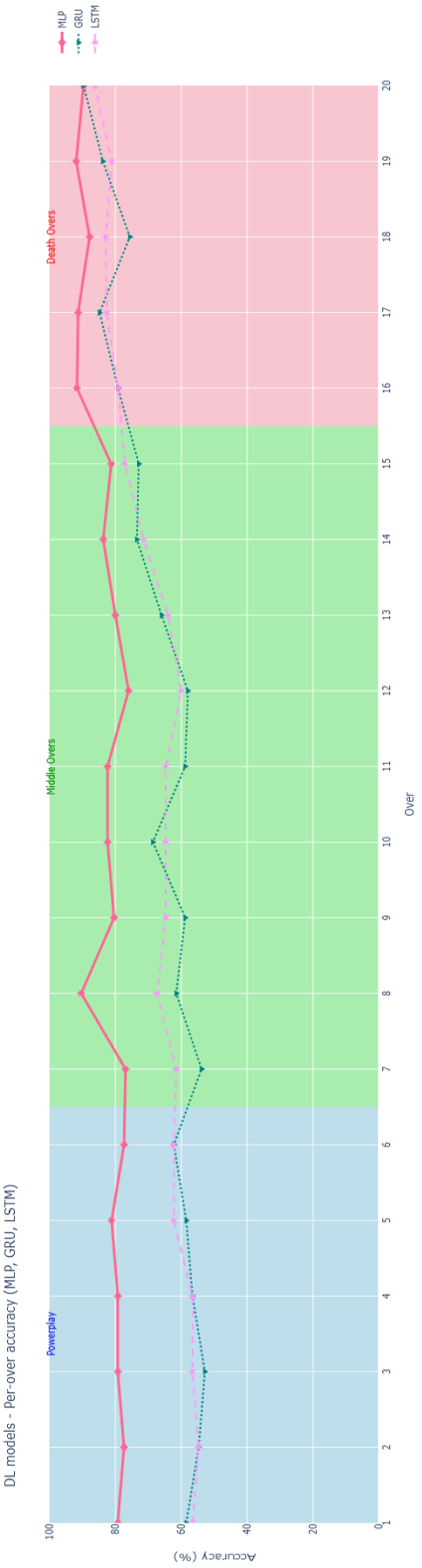

Figure 4.5: Per-over accuracy comparison for deep learning models

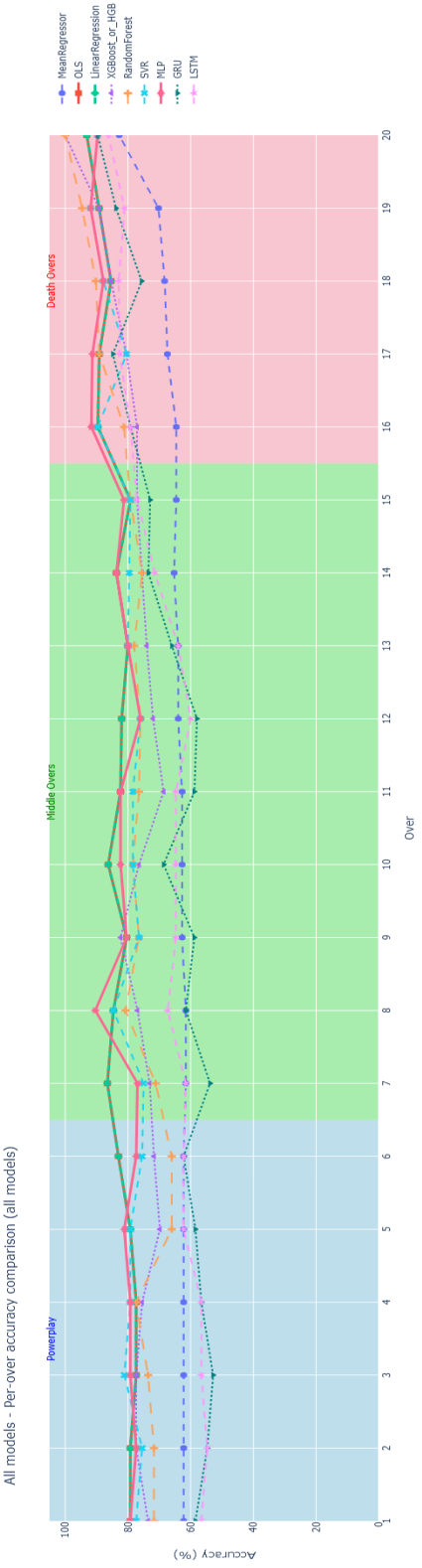

Figure 4.6: Per-over accuracy comparison across all nine models

The predictive models were evaluated to test whether the engineered match-state variables alone are sufficient to explain the outcome of a chase. The evaluation begins with statistical models as the clearest baseline, as they are grounded in direct relationships between required runs, balls remaining, wickets lost, and the eventual result.

Across all data splits, the statistical models performed the best overall. They showed strong generalization and excellent calibration. Ordinary Least Squares and Linear Regression produced the exact same outputs. This confirms that the relationship learned by these methods is stable and highly linear.

Figure 4.1 displays the ROC curves to show how well each model discriminates. There is a clear gap between the predictive models and the baseline. The Mean Regressor stays right on the diagonal with an AUC of 0.5000 across all splits. This makes sense because it ignores all match features completely.

The linear models reached an AUC of 0.8821 on the training data, which rose to 0.9312 on the validation set and settled at 0.8870 on the test set. The close gap between the training and testing metrics proves these simple models generalize perfectly without overfitting.

Table 4.1 confirms that the linear models set a very strong benchmark. They achieved lower MSE and RMSE values. This shows their predicted probabilities track closely to the actual match outcomes. Their log loss values stayed stable between 0.43 and 0.46. This means the models are both accurate and well calibrated.

Figure 4.2 plots the accuracy per over. It shows that the statistical models perform well right from the Powerplay. Their dominance only grows as the innings progresses. The accuracy climbs sharply during the death overs. At this late stage, the match state is highly constrained and the result heavily depends on the remaining resources.

A straight line model cannot represent everything. Machine learning approaches were added to capture complex interactions. This is important because the value of a wicket changes depending on the current over, the target, and the scoring pressure. A high required run rate is manageable with wickets in hand but becomes fatal in the final overs.

Table 4.1 also reveals a major problem within the machine learning models. The tree models suffered from severe overfitting. XGBoost and Random Forest both hit a perfect 1.0000 ROC AUC and 99.96 percent accuracy on the training data. Predictably, their performance dropped sharply on the validation and test sets.

XGBoost struggled the most. It returned an inflated test log loss of 0.9283. This indicates it made highly confident but completely incorrect probability guesses. Support Vector Regression proved to be a much better option. SVR maintained a steady AUC trajectory and secured the lowest MSE among the machine learning models.

Figure 4.3 shows how the three machine learning models behave over time. All of them improve as the innings heads toward the death overs. SVR is the most consistent option through the middle overs. Random Forest catches up strongly at the very end. XGBoost is noticeably uneven and fluctuates during the middle phase.

To push the complexity further, deep learning models were brought in. These networks are built to evaluate the sequential flow of a cricket chase. They learn from the chronological sequence of events rather than just an isolated snapshot.

This makes them ideal for tracking momentum shifts and pressure buildup. The Multi Layer Perceptron acts as a basic neural baseline. Meanwhile, the GRU and LSTM models introduce explicit memory cells to track the history of the game.

The deep learning results in Table 4.1 highlight the efficacy of regularization and early stopping. Unlike the tree-based machine learning models, the neural networks avoided

extreme overfitting on the training set, maintaining steady training AUCs between 0.88 and 0.90. On the validation set, the recurrent networks proved to be exceptionally well-calibrated, with the LSTM producing the lowest log loss (0.3731). On the test set, MLP retained the best hard-classification accuracy (77.01%) and ROC AUC (0.8638) among the neural tier. However, GRU yielded the lowest log loss, proving that recurrent networks are highly effective when the objective is stable, sequence-aware probabilistic forecasting.

Figure 4.4 visualizes the training behavior. The MLP exhibits the smoothest trajectory, with both training and validation loss decreasing steadily. The GRU and LSTM models reduce training loss more aggressively, but their validation loss begins to decouple slightly in the middle epochs.

These diagnostic results confirm an important point about sequential models. They are highly capable of mapping deep patterns over time, but they are very sensitive to overtraining. This completely justifies using the early stopping mechanisms built into the pipeline.

Figure 4.5 hows the accuracy of each neural model over the course of the innings. The MLP performs the best during the Powerplay and the middle overs. The GRU and LSTM models start out a bit slower. They improve steadily as the chase enters the Death Overs. This proves that remembering the sequence of previous deliveries becomes incredibly valuable as the match gets tighter.

By overs 16 to 20, the performance gap between the models disappears completely. At this late stage, the outcome mostly comes down to just the remaining balls and required runs.

Bringing all these findings together, Figure 4.6 provides the clearest picture of the entire modeling setup. The statistical models like OLS and Linear Regression stay very

stable during the middle overs. They prove to be the strongest overall predictors.

The machine learning models sit just behind them. SVR and Random Forest quickly close the gap during the death overs.

The deep learning models definitely remain competitive. They offer unique insights by tracking the actual sequence of events, even if they do not beat the basic statistical models in pure accuracy.

Across every category, the middle overs are clearly the hardest phase to predict. Finally, the death overs force all the models to converge because the end of the match becomes highly predictable.

4.2 DISCUSSION

Results of this dissertation shows that win probability can be modeled with a really good accuracy, and is true for various models that were implemented.

Looking at the best metrics across the 3 categories highlight some clear winners. In statistical group, OLS and Linear Regression offered the best base for generalization as they achieved a good ROC AUC of 0.8870 and accuracy of 81.18% on the test set of the data. In the ML models, SVR stood out the most as it secured MSE of 0.1468 with an accuracy of 79.36%. Finally in the DL models, MLP reached best test AUC of 0.8638. However, LSTM and GRU models also performed wellin probability estimation. LSTM produced validation log loss of 0.3731 and GRU achieved best recurrent log loss at 0.5020.

One major takeaway from this evaluation is that simple linear models perform competitively compared to other categories. This aligns with findings that linear methods remain highly robust in limited-overs cricket compared to more complex ensemble approaches [35]. Linear methods have been shown to provide a better, stable foundation

for score estimation, with dynamic logistic regression which further demonstarted strong performance in settings such as the IPL [13, 10].

Testing nonlinear interactions through the machine learning models exposed a major issue in current sports forecasting. There is a severe danger of overfitting the data.

Random Forest and XGBoost both hit a perfect 1.0000 ROC AUC and 99.96% accuracy during training. However, their performance dropped heavily on the test set. XGBoost returned a highly inflated log loss of 0.9283. The strict chronological data split proves that these near perfect training scores likely come from target leakage. The models were not actually learning general match dynamics. This aligns with findings that, although Random Forest models often achieve a much higher accuracy, simpler regression-based approaches tend to produce more reliable probability estimates [29].

The effectiveness of Support Vector Regression (SVR) further highlights the importance of quantifying match scenarios rather than relying only on overly complex or overfitted decision rules [12].

The sequential deep learning approaches further support prior findings that recurrent networks are better and effective at capturing time-based context in run chases [34, 36].

The distinction between raw classification accuracy and well-calibrated probability estimates is crucial in professional sports analytics, where proper calibration is often more important than predictive accuracy alone [5].

Traditional methods like the Duckworth–Lewis–Stern (DLS) model offer a reliable framework for setting revised targets in the game of cricket [8]. However, these approaches are primarily designed for static scenarios and therefore fail to capture the fluctuations, momentum shifts, and dynamic contextual changes that occur throughout a live match.

To understand exactly why certain models systematically beat others in specific areas,

we must now take a look at how they are built and used. Now we will discuss why certain models outperformed others:

- **Why OLS outperformed Tree-Ensembles:** OLS can sometimes generalize better than very flexible models because it follows a strict linear form. In this dataset, that can actually be an advantage. T20 ball by ball data is noisy, and tree based models like XGBoost and Random Forest can get pulled too much by small delivery level changes. As a result, they may fit the training data very closely instead of learning the broader pattern of a chase.

- **Why SVR outperformed Tree-Ensembles:** SVR performed better than the tree ensembles because it was better at capturing the overall shape of the chase. Its epsilon tube makes it less sensitive to small fluctuations in individual deliveries. That helped it focus on the wider match trajectory rather than reacting too strongly to every minor change in runs or wickets.

- **Why LSTM and GRU outperformed MLP and OLS in Calibration:** LSTM and GRU were stronger than MLP and OLS for calibration because they use sequence information. OLS and MLP treat each delivery more like a separate input, so they do not naturally capture momentum. A chasing innings is not just a set of isolated balls. It has rhythm, pressure shifts, and sudden turns. GRU and LSTM are better at learning that pattern because they keep memory of earlier deliveries and use that context to judge what comes next. This makes them better at telling the difference between a controlled chase and one that is starting to fall apart.

Overall, by focusing on clean data preparation and sequence based modeling rather than player specific features [32], this framework gives more reliable and context aware

win probability for a T20 World Cup chase.

4.3 CONCLUSION

This research in the end has developed a reproducible ball by ball win probability pipeline designed for Men's T20 World Cup matches by processing 17 years of historical match data from 2007-2025, the study provides a simple evaluation of how different models deal with dynamic nature of T20 format.

The main conclusion is that win probability isn't just a static snapshot of the current score, it also reflects the momentum built up inside the innings over time. Statistical models such as OLS with an AUC of 0.9309, while at the same time, sequential DL models gave most useful insight for real world use.

Mostly GRU and LSTM models showed a strong ability to capture the ongoing match rhytm. They produced smooth probability changes that matched well with the pressure normally felt by players and spectators during a chase. The results show that linear features are very much predicitve, but more complex models are also needed to provide better calibrations which is required for professional sports analytics.

A major contribution of this work was to focus on data engineering at the base of the models predictions. The custom match ID system and proper correction of the dataset and cleaning solved several structural issues which would have otherwise affected quality of the dataset. This technical care matters as the models learn from an accurate representation of the game, rather than from data errors or dataset anomalies.

4.3.1 Significance of Findings

The results suggest that in a T20 World Cup chase, the relationship between scoring pressure and resource depletion is so strong that even classical models can achieve high

accuracy. Sequential models then add the extra detail needed for professional quality performance. This pipeline not only performs better than simple run rate methods, but also produces smooth, real time probabilities that can be used in modern web applications and broadcasting tools [26, 54].

The findings also show an important trade off between discrimination and calibration in sports forecasting. Some models were very good at identifying the eventual winner, but the LSTM model was better at giving a trustworthy probability that reflected the actual chance of winning. This difference is important for decision support tools for coaches and team managers, where calibrated probabilities are more useful than a simple yes or no prediction.

Overall, the performance of this multi category models shows that a category modeling approach is the most effective way to compare performance across different levels of model complexity.

4.3.2 Limitations

- **Exclusion of Player-Specific Metrics:**One major limitation of this work is leaving out player specific statistics/ attributes. I chose to do this on purpose so that the models focus strictly on match state variables like runs and wickets and this prevents the curse of dimensionality and stops the models from overfitting on the dataset and it ensures the win probabilities come from objective match data rather than just relying on a player's reputation.

- **Omission of Afghanistan Matches:** I also had to exclude Afghanistan matches [2].This reduces the total size of the dataset. Since these were fully completed games, including them would have made the data more comprehensive and it could

have improved the overall performance of the models.

- **Absence of Environmental and Pitch Factors:** The current system also ignores environmental and pitch factors. It relies entirely on mathematical match data. It does not account for pitch wear, boundary sizes, or dew. Dew is a massive factor in T20 cricket, especially in the second innings. It makes the ball hard to grip and puts the fielding side at a severe disadvantage. This directly shifts the true win probability. Leaving these physical conditions out limits how well the models can properly do ball by ball probability.

4.3.3 Future Work

Even though the current pipeline is successful, there are several ways to improve the research, accuracy and depth of the win probability for chasing team:

- **Integration of Player Specific Metrics:** Future updates to the pipeline should try to include real time player data. The current system only uses basic match variables. Adding things like a batter's career strike rate or a bowler's recent form would be highly valuable. This would let the models account for individual skill levels. The challenge will be doing this without making the feature space too complicated.

- **Continuous Dataset Expansion:** The flexible code architecture built for this study is ready to accept new data. It can easily pull in matches from the upcoming 2026 T20 World Cup. As the Cricsheet database grows, analysts can keep fine tuning these models on the latest playing styles. This ensures the system stays relevant as the sport changes.

- **Live Dashboard and Visualization:** A logical next step for this research is building a live dashboard. A web application could use the ball time proxy system to track probabilities during live matches. This would give fans and broadcasters a real time view of match momentum.

Ultimately, this dissertation sets up a strong foundation for modern cricket analytics. It combines data engineering with deep learning. The project proves that tracking win probability ball by ball helps us understand the complex nature of the T20 World Cup. It provides a more better and more clear path for the future for more accurate sports forecasting.

REFERENCES

[1] Cricsheet. *T20s Male (Matches) Dataset*. 2025. URL: https://cricsheet.org/downloads/t20s_male.zip (visited on 07/31/2025).

[2] Cricsheet. *Explanation for withholding of Afghanistani matches*. 2024. URL: https://cricsheet.org/article/explanation-for-withholding-of-afghanistani-matches/ (visited on 07/31/2025).

[3] Stephen R Clarke. "Dynamic programming in one-day cricket-optimal scoring rates". In: *Journal of the Operational Research Society* 39.4 (1988), pp. 331–337. DOI: https://doi.org/10.1057/jors.1988.60.

[4] Adriano K Suzuki et al. "A Bayesian approach for predicting match outcomes: the 2006 (Association) Football World Cup". In: *Journal of the Operational Research Society* 61.10 (2010), pp. 1530–1539. DOI: https://doi.org/10.1057/jors.2009.127.

[5] Chancellor Johnstone and Dan Nettleton. "Using conformal win probability to predict the winners of the canceled 2020 NCAA basketball tournaments". In: *The American Statistician* 78.3 (2024), pp. 304–317. DOI: https://doi.org/10.1080/00031305.2023.2283199.

[6] Rahul Baboota and Harleen Kaur. "Predictive analysis and modelling football results using machine learning approach for English Premier League". In: *International Journal of Forecasting* 35.2 (2019), pp. 741–755. DOI: https://doi.org/10.1016/j.ijforecast.2018.01.003.

[7] Sohail Akhtar and Philip Scarf. "Forecasting test cricket match outcomes in play". In: *International Journal of Forecasting* 28.3 (2012), pp. 632–643. DOI: https://doi.org/10.1016/j.ijforecast.2011.08.005.

[8] M Asif and IG McHale. "A generalized non-linear forecasting model for limited overs international cricket". In: *International journal of forecasting* 35.2 (2019), pp. 634–640. DOI: https://doi.org/10.1016/j.ijforecast.2018.12.003.

[9] Muhammad Asif and Ian G McHale. "In-play forecasting of win probability in One-Day International cricket: A dynamic logistic regression model". In: *International Journal of Forecasting* 32.1 (2016), pp. 34–43. DOI: https://doi.org/10.1016/j.ijforecast.2015.02.005.

[10] Sonish Lamsal and David Kahle. "In-Game Win Prediction Models for Cricket". In: *Southwest Data Science Conference*. Springer. 2024, pp. 148–168. DOI: https://doi.org/10.1007/978-3-031-67871-4_11.

[11] Subrat Sarangi and RK Renin Singh. "Winning one-day international cricket matches: A cross-team perspective". In: *Journal of Business Analytics* 6.1 (2023), pp. 39–58. DOI: https://doi.org/10.1080/2573234X.2022.2041370.

[12] R Raja Subramanian et al. "Regression to Forecast: An In-Play Outcome Prediction for One-Day Cricket Matches". In: *Proceedings of the International Conference on Cognitive and Intelligent Computing: ICCIC 2021, Volume 2*. Springer. 2023, pp. 43–55. DOI: https://doi.org/10.1007/978-981-19-2358-6_5.

[13] Tejinder Singh, Vishal Singla, and Parteek Bhatia. "Score and winning prediction in cricket through data mining". In: *2015 international conference on soft computing techniques and implementations (ICSCTI)*. IEEE. 2015, pp. 60–66. DOI: 10.1109/ICSCTI.2015.7489605.

[14] Fahim Ahmed Shakil et al. "Predicting the result of a cricket match by applying data mining techniques". In: *Proceedings of the Computational Methods in Systems and Software*. Springer, 2020, pp. 758–770. DOI: https://doi.org/10.1007/978-3-030-63319-6_70.

[15] Shristi Priya et al. "Analysis and winning prediction in t20 cricket using machine learning". In: *2022 Second International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*. IEEE. 2022, pp. 1–4. DOI: https://doi.org/10.1109/ICAECT54875.2022.9807929.

[16] Praffulla Kumar Dubey, Harshit Suri, and Saurabh Gupta. "Naıve Bayes algorithm based match winner prediction model for T20 cricket". In: *Intelligent computing and applications: Proceedings of ICICA 2019*. Springer, 2020, pp. 435–446. DOI: https://doi.org/10.1007/978-981-15-5566-4_38.

[17] G Suseela et al. "Predictive Analysis of Cricket Match Outcomes: A Comparative Study of Machine Learning and Data Mining Techniques". In: *International Conference on Hybrid Intelligent Systems*. Springer. 2023, pp. 425–432. DOI: https://doi.org/10.1007/978-3-031-78925-0_42.

[18] Soumya Ranjan Mahanta and Mrutyunjaya Panda. "Sports Prediction for Cricket Match Using Grid Search and Extreme Gradient Boosting Classifier". In: *International Conference on Machine Learning, IoT and Big Data*. Springer. 2024, pp. 159–177. DOI: https://doi.org/10.1007/978-981-97-8160-7_13.

[19] Anjali Singhal et al. "IPL Analysis and Match Prediction". In: *Intelligent System Design: Proceedings of INDIA 2022*. Springer, 2022, pp. 29–38. DOI: https://doi.org/10.1007/978-981-19-4863-3_3.

[20] Poulomi Paul, Pratyay Ranjan Datta, and Ashutosh Kar. "Classification model to predict the outcome of an IPL match". In: *Analytics Global Conference*. Springer. 2023, pp. 83–111. DOI: https://doi.org/10.1007/978-3-031-50815-8_6.

[21] Abdul Basit et al. "ICC T20 Cricket World Cup 2020 winner prediction using machine learning techniques". In: *2020 IEEE 23rd International Multitopic Conference (INMIC)*. IEEE. 2020, pp. 1–6. DOI: 10.1109/INMIC50486.2020.9318077.

[22] Md All Shahoriar Tonmoy et al. "A data-driven approach to predict scores in T20 cricket match using machine learning classifier". In: *International conference on big data, IoT and machine learning*. Springer. 2023, pp. 727–745. DOI: https://doi.org/10.1007/978-981-99-8937-9_49.

[23] Amal Kaluarachchi and S Varde Aparna. "CricAI: A classification based tool to predict the outcome in ODI cricket". In: *2010 Fifth International Conference on Information and Automation for Sustainability*. IEEE. 2010, pp. 250–255. DOI: 10.1109/ICIAFS.2010.5715668.

[24] Maheswari Subburaj et al. “Artificial intelligence for smart in match winning prediction in Twenty20 cricket league using machine learning model”. In: *Artificial Intelligence for Smart Healthcare*. Springer, 2023, pp. 31–46. DOI: https://doi.org/10.1007/978-3-031-23602-0_3.

[25] Antony Anuraj et al. “Sports data mining for cricket match prediction”. In: *International Conference on Advanced Information Networking and Applications*. Springer. 2023, pp. 668–680. DOI: https://doi.org/10.1007/978-3-031-28694-0_63.

[26] Eeshan Mundhe, Ishan Jain, and Sanskar Shah. “Live cricket score prediction web application using machine learning”. In: *2021 International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON)*. IEEE. 2021, pp. 1–6. DOI: 10.1109/SMARTGENCON51891.2021.9645855.

[27] DGTL Karunathilaka et al. ““Can Mumbai Indians Chase the Target?”: Predict the Win Probability in IPL T20-20”. In: *Proceedings of Sixth International Congress on Information and Communication Technology: ICICT 2021, London, Volume 2*. Springer. 2021, pp. 991–999. DOI: https://doi.org/10.1007/978-981-16-2380-6_88.

[28] Taufiq Al Hasib Sadi et al. “Cric Win: Cricket Bigdata Processing with a Deep MLP Classifier to Predict Direct Win in Live Games”. In: *ICC 2024-IEEE International Conference on Communications*. IEEE. 2024, pp. 7–12. DOI: https://doi.org/10.1109/ICC51166.2024.10622946.

[29] Aryan Satwani et al. “Live cricket predictions for runs and win using machine learning”. In: *2024 IEEE 12th International Conference on Intelligent Systems (IS)*. IEEE. 2024, pp. 1–6. DOI: https://doi.org/10.1109/IS61756.2024.10705216.

[30] Md Taufiq Al Hasib Sadi et al. “Deep learning to reliable score prediction in hundred ball cricket matches”. In: *2023 26th International Conference on Computer and Information Technology (ICCIT)*. IEEE. 2023, pp. 1–6. DOI: https://doi.org/10.1109/ICCIT60459.2023.10441388.

[31] Sayed Sajid Ali et al. “Cricket Score Prediction using Machine Learning”. In: *2025 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC)*. IEEE. 2025, pp. 1–6. DOI: https://doi.org/10.1109/ASSIC64892.2025.11158251.

[32] U Vishal Raj et al. “Cricket Score Prediction using Player-Specific Performance and Dynamic Metrics”. In: *International Conference on Intelligent Systems and Digital Transformation (ICISD 2025)*. Atlantis Press. 2025, pp. 613–628. DOI: https://doi.org/10.2991/978-94-6463-866-0_51.

[33] Vemula Vivek Siddharth et al. “Cricket Forecast: Unraveling Future Matches’ Outcomes”. In: *International Conference on Computational Intelligence in Data Science*. Springer. 2024, pp. 353–364. DOI: https://doi.org/10.1007/978-3-031-69986-3_27.

[34] Natwar Modani et al. “Predicting Outcomes in Limited-Overs Cricket Matches”. In: *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*. 2020, pp. 65–72. DOI: https://doi.org/10.1145/3371158.3371166.

[35] Md Sabbir Hossain et al. “One day international cricket match score prediction using machine learning approaches”. In: *2024 IEEE international conference on smart power control and renewable energy (ICSPCRE)*. IEEE. 2024, pp. 1–6. DOI: 10.1109/ICSPCRE62303.2024.10674897.

[36] P Kumar, N Karthikeyan, et al. “Predicting the Results of Cricket Matches Using Deep Learning Method”. In: *2024 International Conference on Smart Technologies for Sustainable Development Goals (ICSTSDG)*. IEEE. 2024, pp. 1–6. DOI: https://doi.org/10.1109/ICSTSDG61998.2024.11026460.

[37] Ganeshapillai Gartheeban and John Guttag. “A data-driven method for in-game decision making in mlb: when to pull a starting pitcher”. In: *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2013, pp. 973–979. DOI: https://doi.org/10.1145/2487575.2487660.

[38] V Sivaramaraju Vetukuri et al. “Analytic for Cricket Match Winner Prediction Through Major Events Quantification”. In: *International Conference on Image Processing and Capsule Networks*. Springer. 2022, pp. 171–181. DOI: https://doi.org/10.1007/978-3-031-12413-6_14.

[39] Trevor Hastie. *The elements of statistical learning: data mining, inference, and prediction*. 2009. DOI: https://doi.org/10.1007/978-0-387-84858-7.

[40] Arthur P Dempster, Martin Schatzoff, and Nanny Wermuth. “A simulation study of alternatives to ordinary least squares”. In: *Journal of the American Statistical Association* 72.357 (1977), pp. 77–91. DOI: https://doi.org/10.1080/01621459.1977.10479910.

[41] Douglas C Montgomery, Elizabeth A Peck, and G Geoffrey Vining. *Introduction to linear regression analysis*. John Wiley & Sons, 2021. DOI: 10.1111/insr.12020_10.

[42] Tianqi Chen and Carlos Guestrin. “Xgboost: A scalable tree boosting system”. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, pp. 785–794. DOI: https://doi.org/10.1145/2939672.2939785.

[43] Leo Breiman. “Random forests”. In: *Machine learning* 45.1 (2001), pp. 5–32. DOI: https://doi.org/10.1023/A:1010933404324.

[44] Harris Drucker et al. “Support vector regression machines”. In: *Advances in neural information processing systems* 9 (1996). URL: https://proceedings.neurips.cc/paper/1996/hash/d38901788c533e8286cb6400b40b386d-Abstract.html.

[45] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536. DOI: https://doi.org/10.1038/323533a0.

[46] Kyunghyun Cho et al. “Learning phrase representations using RNN encoder–decoder for statistical machine translation”. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014, pp. 1724–1734. DOI: 10.3115/v1/D14-1179.

[47] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[48] Timothy Hodson, Thomas Over, and Sydney Foks. "Mean Squared Error, Deconstructed". In: *Journal of Advances in Modeling Earth Systems* 13 (Dec. 2021). DOI: 10.1029/2021MS002681.

[49] Timothy Hodson. "Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not". In: *Geoscientific Model Development* 15 (July 2022), pp. 5481–5487. DOI: 10.5194/gmd-15-5481-2022.

[50] "Coefficient of Determination". In: *The Concise Encyclopedia of Statistics*. New York, NY: Springer New York, 2008, pp. 88–91. ISBN: 978-0-387-32833-1. DOI: 10.1007/978-0-387-32833-1_62.

[51] Andréa Puzzi Nicolau et al. "Accuracy Assessment: Quantifying Classification Quality". In: *Cloud-Based Remote Sensing with Google Earth Engine: Fundamentals and Applications*. Ed. by Jeffrey A. Cardille et al. Cham: Springer International Publishing, 2024, pp. 135–145. ISBN: 978-3-031-26588-4. DOI: 10.1007/978-3-031-26588-4_7.

[52] Vladimir Vovk. "The Fundamental Nature of the Log Loss Function". In: *Lecture Notes in Computer Science* (Feb. 2015). DOI: 10.1007/978-3-319-23534-9_20.

[53] Francisco Melo. "Area under the ROC Curve". In: *Encyclopedia of Systems Biology*. Ed. by Werner Dubitzky et al. New York, NY: Springer New York, 2013, pp. 38–39. ISBN: 978-1-4419-9863-7. DOI: 10.1007/978-1-4419-9863-7_209.

[54] R Suguna et al. "Utilizing machine learning for sport data analytics in cricket: score prediction and player categorization". In: *2023 IEEE 3rd Mysore Sub Section International Conference (MysuruCon)*. IEEE. 2023, pp. 1–6. DOI: 10.1109/MysuruCon59703.2023.10396955.